



Proyecto #1

Compiladores e intérpretes

Análisis Léxico y Sintáctico

Estudiantes:

Tayler Wynta Rodríguez | 2024143103

Maikel Flores Navarro | 2024148346

Profesor:

Ing. Allan Rodriguez Davila

II Semestre 2026

Contenidos

Manual de usuario	3
Instrucciones de compilación, ejecución y uso.	3
Pruebas de funcionalidad	4
Descripción del problema.	9
Diseño del programa: decisiones de diseño, algoritmos usados.	10
Librerías usadas	12
Análisis de resultados: objetivos alcanzados, objetivos no alcanzados, y razones por las cuales no se alcanzaron los objetivos (en caso de haberlos).	12
Justificación de toma de decisiones: por qué se generaron las producciones en expresiones y la precedencia.	13
Bitácora.....	15

Manual de usuario

Instrucciones de compilación, ejecución y uso.

Desde la raíz del proyecto (Proyecto1/), en una terminal:

1. Generar Parser.java y sym.java desde la gramática:

```
java -jar lib\jflex-full-1.9.1.jar --encoding UTF-8 -d src especificaciones\Lexer.flex
```

```
java -cp lib\java-cup-11b-20151001.jar java_cup.Main -destdir src -parser Parser -symbols sym especificaciones\Parser.cup
```

2. Compilar

```
New-Item -ItemType Directory -Force out
```

```
javac -encoding UTF-8 -cp lib\java-cup-runtime-11b-20151001.jar -d out src>Main.java src\Lexer.java src\Parser.java src\sym.java src\TablaSimbolos.java
```

Ejecución

Asegurarse de que exista la carpeta salida/

(si no existe, crearla, el programa no la crea solo).

Desde la raíz del proyecto:

```
java -cp "out;lib\java-cup-runtime-11b-20151001.jar" Main
```

El -Dfile.encoding=UTF-8 es necesario porque el lenguaje usa símbolos especiales (F, λ , θ , Σ , ϵ , $\varnothing$, $\int$, γ , etc.) y sin forzar la codificación se pueden leer mal según la configuración.

Uso

El programa analiza el archivo pruebas/Prueba.txt

- Imprime en consola cada token encontrado y, al final, si el archivo pertenece o no a la gramática.
- Escribe salida/tokens.txt con cada token, su lexema y la línea donde apareció.
- Escribe salida/errores_lexicos.txt con los errores léxicos encontrados (si hubo).
- Escribe salida/errores_sintacticos.txt con los errores sintácticos encontrados (si hubo).

Si el archivo no pertenece a la gramática, el programa no se detiene en el primer error, sigue revisando el resto y reporta todos los que encuentre en esa misma corrida.

Pruebas de funcionalidad

Ejecutando en consola el comando para crear el lexer, compila correctamente sin errores.

```
PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1> java -jar lib\jflex-full-1.9.1.jar --encoding UTF-8 -d src especificaciones\Lexer.flex
Reading "especificaciones\Lexer.flex"
Constructing NFA : 349 states in NFA
Converting NFA to DFA :
.....
158 states before minimization, 146 states in minimized DFA
Old file "src\Lexer.java" saved as "src\Lexer.java~"
Writing code to "src\Lexer.java"
PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1>
```

Ejecutando en consola el comando para crear el parser, compila correctamente sin errores.

```
PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1> java -cp lib\java-cup-11b-20151001.jar java_cup.Main -destdir src -parser Parser -symbols sym especificaciones\Parser.cup
----- CUP v0.11b 20150930 (SVN rev 66) Parser Generation Summary -----
0 errors and 0 warnings
52 terminals, 62 non-terminals, and 154 productions declared,
producing 329 unique parse states.
0 terminals declared but not used.
0 non-terminals declared but not used.
0 productions never reduced.
0 conflicts detected (0 expected).
Code written to "Parser.java", and "sym.java".
----- (CUP v0.11b 20150930 (SVN rev 66))
PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1>
```

Al momento de compilar ambos en un solo archivo para que la lógica implementada para el análisis completo se realice, lo hace de forma compatible y sin errores.

```
----- (CUP v0.11b 20150930 (SVN rev 66))
PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1> javac -encoding UTF-8 -cp lib\java-cup-runtime-11b-20151001.jar -d out src\Main.java src\Lexer.java src\Parser.java src\sym.java src\TablaSimbolos.java
Note: src\Main.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1>
```

Válido: En este caso se ejecutó la prueba con el léxico y sintáxis válida, además de haber mostrado previamente todos los tokens del código.

```

PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1> java -cp "out;lib\java-cup-runtime-11b-20151001.jar" Main
Token: VAL | Lexema: val | Linea: 8
Token: INT | Lexema: int | Linea: 8
Token: ID | Lexema: contadorGlobal | Linea: 8
Token: ASIGNACION | Lexema: ? | Linea: 8
Token: LIT_INT | Lexema: 0 | Linea: 8
Token: FIN_EXPR | Lexema: » | Linea: 8
Token: VAL | Lexema: val | Linea: 9
Token: INT | Lexema: int | Linea: 9

Token: ID | Lexema: sensores | Linea: 348
Token: PAR_INI | Lexema: ?: | Linea: 348
Token: LIT_FLOAT | Lexema: 2.5 | Linea: 348
Token: COMA | Lexema: , | Linea: 348
Token: LIT_FLOAT | Lexema: 3.5 | Linea: 348
Token: PAR_FIN | Lexema: :? | Linea: 348
Token: FIN_EXPR | Lexema: » | Linea: 348
Token: BLOQUE_FIN | Lexema: :? | Linea: 352

Analisis lexico finalizado.

Analisis sintactico finalizado.
Tabla de simbolos guardada en: salida/tabla_simbolos.txt

El programa pertenece a la gramatica.
PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1>

```

Tabla de símbolos

1	=== TABLA DE SIMBOLOS: Global ===			
2	Nombre	Tipo	Categoria	Linea
3	-----	----	-----	-----
4	contadorGlobal	int	variable	8
5	limiteGlobal	int	variable	9
6	temperaturaGlobal	float	variable	10
7	voltajeGlobal	float	variable	11
8	sistemaActivo	bool	variable	12
9	modoSeguro	bool	variable	13
10	estadoGlobal	char	variable	14
11	nombreSistema	string	variable	15
12	numeros	int[]	arreglo	19
13	valores	int[]	arreglo	20
14	voltajes	float[]	arreglo	21
15	obtenerLimite	int	funcion	33
16	duplicar	int	funcion	47
17	sumar	int	funcion	59
18	calcularVoltaje	float	funcion	67
19	compararValores	bool	funcion	75
20	operaciones	int	funcion	87
21	clasificar	int	funcion	114
22	contar	int	funcion	147
23	recorrer	int	funcion	167
24	trabajarArreglo	int	funcion	187
25	validar	bool	funcion	207
26	invertir	bool	funcion	221
27	solicitarNumero	int	funcion	233
28	buscarLimite	int	funcion	251
29	sensores	float	funcion	275

Tokens generados

```

1 Token: VAL | Lexema: val | Linea: 8      Maikel Flores,
2 Token: INT | Lexema: int | Linea: 8
3 Token: ID | Lexema: contadorGlobal | Linea: 8
4 Token: ASIGNACION | Lexema: = | Linea: 8
5 Token: LIT_INT | Lexema: 0 | Linea: 8
6 Token: FIN_EXPR | Lexema: » | Linea: 8
7 Token: VAL | Lexema: val | Linea: 9
8 Token: INT | Lexema: int | Linea: 9
9 Token: ID | Lexema: limiteGlobal | Linea: 9
10 Token: ASIGNACION | Lexema: = | Linea: 9
11 Token: LIT_INT | Lexema: 25 | Linea: 9
12 Token: FIN_EXPR | Lexema: » | Linea: 9
13 Token: VAL | Lexema: val | Linea: 10
14 Token: FLOAT | Lexema: float | Linea: 10
15 Token: ID | Lexema: temperaturaGlobal | Linea: 10
16 Token: ASIGNACION | Lexema: = | Linea: 10
17 Token: LIT_FLOAT | Lexema: 23.5 | Linea: 10
18 Token: FIN_EXPR | Lexema: » | Linea: 10
19 Token: VAL | Lexema: val | Linea: 11
20 Token: FLOAT | Lexema: float | Linea: 11
21 Token: ID | Lexema: voltajeGlobal | Linea: 11
22 Token: ASIGNACION | Lexema: = | Linea: 11
23 Token: LIT_FLOAT | Lexema: 3.3 | Linea: 11
24 Token: FIN_EXPR | Lexema: » | Linea: 11
25 Token: VAL | Lexema: val | Linea: 12
26 Token: BOOL | Lexema: bool | Linea: 12
27 Token: ID | Lexema: sistemaActivo | Linea: 12
28 Token: ASIGNACION | Lexema: = | Linea: 12

```

Inválido: Este es un caso de prueba con el léxico y sintaxis inválida, además de haber mostrado previamente todos los tokens del código.

```

1 void principal €::» {
2     val int numero = 5 »
3     write €::» »
4     val int numero =
5     @
6     @
7     @
8     :?

```

Tokens generados

```

Maikel Flores, last week | author (Maikel Flores)
1 Token: VOID | Lexema: void | Línea: 1
2 Token: PRINCIPAL | Lexema: principal | Línea: 1
3 Token: PAR_INI | Lexema: €: | Línea: 1
4 Token: PAR_FIN | Lexema: :ə | Línea: 1
5 Token: BLOQUE_INI | Lexema: ¿: | Línea: 1
6 Token: VAL | Lexema: val | Línea: 2
7 Token: INT | Lexema: int | Línea: 2
8 Token: ID | Lexema: numero | Línea: 2
9 Token: ASIGNACION | Lexema: † | Línea: 2
10 Token: LIT_INT | Lexema: 5 | Línea: 2
11 Token: FIN_EXPR | Lexema: » | Línea: 2
12 Token: WRITE | Lexema: write | Línea: 3
13 Token: PAR_INI | Lexema: €: | Línea: 3
14 Token: PAR_FIN | Lexema: :ə | Línea: 3
15 Token: FIN_EXPR | Lexema: » | Línea: 3
16 Token: VAL | Lexema: val | Línea: 4
17 Token: INT | Lexema: int | Línea: 4
18 Token: ID | Lexema: numero | Línea: 4
19 Token: ASIGNACION | Lexema: † | Línea: 4
20 Token: FIN_EXPR | Lexema: » | Línea: 4

```

Tabla de símbolos generada

1	=== TABLA DE SIMBOLOS: Global ===			
2	Nombre	Tipo	Categoría	Línea
3	-----	----	-----	-----
4	(sin símbolos)			
5				
6	=== TABLA DE SIMBOLOS: Global / principal ===			
7	Nombre	Tipo	Categoría	Línea
8	-----	----	-----	-----
9	numero	int	variable	2

```
Error lexico en linea 5: caracter no reconocido '@'
Error lexico en linea 6: caracter no reconocido '@'
Error lexico en linea 7: caracter no reconocido '@'
Token: BLOQUE_FIN | Lexema: :? | Línea: 8
```

Analisis lexico finalizado.

[ERROR SINTACTICO] Línea 3: token inesperado -> ':?'

[ERROR SINTACTICO] Línea 4: token inesperado -> '»'

Analisis sintactico finalizado.

Tabla de simbolos guardada en: salida/tabla_simbolos.txt

El programa NO pertenece a la gramatica.

Ver errores lexicos en: salida/errores_lexicos.txt

Ver errores sintacticos en: salida/errores_sintacticos.txt

PS C:\Users\Tayle\Documents\GitHub\Proyecto-1-Compiladores\Compiladores\Proyecto1>

Compiladores > Proyecto1 > salida >  errores_lexicos.txt

```
1 Error lexico en linea 5: caracter no reconocido '@'
2 Error lexico en linea 6: caracter no reconocido '@'
3 Error lexico en linea 7: caracter no reconocido '@'
4
```

Compiladores > Proyecto1 > salida >  errores_sintacticos.txt

```
1 [ERROR SINTACTICO] Línea 3: token inesperado -> ':?'
2 [ERROR SINTACTICO] Línea 4: token inesperado -> '»'
3
```

Descripción del problema.

La idea es armar un lenguaje de programación nuevo, pensado para configurar chips, algo liviano, pensado para sistemas embebidos, donde no se necesita todo lo que trae un lenguaje grande sino algo simple y directo para hacer operaciones básicas.

Como es un lenguaje que no existe todavía, no alcanza con solo definirlo en papel: hay que construir las primeras dos fases de un compilador para él. Primero el analizador léxico, que lee el código fuente carácter por carácter y lo agrupa en tokens (palabras reservadas, identificadores, números, operadores, etc.), detectando si hay algo que no se puede reconocer. Y después el analizador sintáctico, que toma esos tokens y verifica si están acomodados de una forma que respete la gramática del lenguaje, es decir, si lo que se escribió realmente "tiene sentido" según las reglas que se definieron.

Las dos fases tienen que trabajar juntas sobre el mismo archivo fuente: leerlo, decir qué tokens encontraron, decir si el archivo completo pertenece o no a la gramática, y si algo está mal, avisar en qué línea está el error, sin que un solo error tumbe todo el análisis, sino que se pueda seguir revisando el resto del archivo.

Diseño del programa: decisiones de diseño, algoritmos usados.

Cómo está armado por dentro

El programa funciona en tres piezas que se van pasando el trabajo. El Lexer lee el archivo carácter por carácter y va sacando los tokens. El Parser toma esos tokens y arma la estructura gramatical. Y Main.java es el que coordina todo: primero corre una pasada léxica que saca tokens.txt y errores_lexicos.txt, y después corre el parser sobre el mismo archivo para sacar errores_sintacticos.txt. Al final revisa si hubo errores en cualquiera de las dos pasadas para decidir si el archivo quedó aceptado o no. No se fija en si algo tiró una excepción, porque resulta que CUP no siempre tira excepción cuando falla, y eso nos hizo tropezar una vez.

El lexer: un autómatas, no un montón de ifs

JFlex no revisa el texto a mano con reglas sueltas. Lo que hace es convertir todas las expresiones regulares del .flex en un autómatas finito determinista (un DFA) y lo minimiza antes de generar el código. En nuestro caso termina con 146 estados después de la minimización. Eso significa que reconocer cualquier token, sin importar qué tan largo sea, siempre toma el mismo trabajo por carácter, sin backtracking ni nada raro.

Para los errores léxicos no detenemos la ejecución ni tiramos excepción. El lexer los va guardando en una cola interna, usando hayErroresPendientes() y obtenerSiguienteError(), y sigue escaneando normal. Así, en una sola corrida se pueden encontrar todos los errores léxicos del archivo, no solo el primero que aparece.

El parser: shift-reduce de abajo hacia arriba

CUP genera un parser LALR(1), que básicamente es una máquina con una pila. Va metiendo tokens hasta que reconoce que lo que tiene arriba de la pila calza con el lado derecho de alguna regla, y en ese momento lo reduce a un no terminal. Decidimos no usar las declaraciones precedence de CUP para resolver las ambigüedades entre operadores. En vez de eso, cada nivel de precedencia quedó como su propio no terminal, así que el autómata que genera CUP nunca tiene que "decidir" entre shift o reduce por una regla de precedencia, porque ya viene resuelto desde la gramática misma.

Recuperación de errores: modo pánico

Cuando el parser se topa con algo que no calza, no aborta todo de una vez. Hay producciones especiales, como `SENTENCIA ::= error FIN_EXPR`, que básicamente le dicen: si no entendiste nada, botá todo lo que venga hasta encontrar un `»` y seguí desde ahí. Eso es modo pánico clásico: se pierde la sentencia que estaba mal, pero el resto del archivo se sigue analizando igual, así que una sola corrida puede terminar reportando varios errores sintácticos en vez de morir apenas encuentra el primero.

Tabla de símbolos: pila de ámbitos

La tabla de símbolos no es una simple lista. Es una pila de ámbitos: cada vez que se entra a una función o a un bloque, se apila un ámbito nuevo, y cuando se sale, se desapila. Cada ámbito guarda sus propios símbolos en una lista, más un `HashSet` aparte que solo tiene los nombres, para poder detectar duplicados rápido sin tener que recorrer toda la lista cada vez que se agrega algo nuevo.

Todo esto se construye en la misma pasada del parser, no en una vuelta aparte. Cada regla de la gramática que declara algo, ya sea una variable, un parámetro o una función, dispara una acción directa dentro del `cup` que llama a `tablaSimbolos.agregarSimbolo()`. Y las reglas que abren o cierran un bloque o una función llaman a `abrirAmbito()` y `cerrarAmbito()`. Por eso, si alguien repite un nombre dentro del mismo ámbito, `agregarSimbolo()` devuelve `false` justo en el momento en que eso pasa, sin tener que esperar a que termine de parsear todo el archivo para darse cuenta.

Librerías usadas

- **JFlex** (jflex-full-1.9.1.jar): genera Lexer.java a partir de Lexer.flex.
- **CUP** (java-cup-11b-20151001.jar): genera Parser.java y sym.java a partir de Parser.cup.
- **CUP runtime** (java-cup-runtime-11b-20151001.jar) : clases Symbol y Scanner que el parser generado necesita para correr.
- **java.io** (FileReader, FileWriter): leer el archivo fuente y escribir tokens.txt / archivos de errores.
- **java.nio.charset.StandardCharsets**: forzar UTF-8 (el lenguaje usa símbolos Unicode).
- **java.util.ArrayDeque**: cola de errores léxicos pendientes, y pila de ámbitos en la tabla de símbolos.
- **java.util.HashSet**: detectar nombres duplicados en cada ámbito.
- **java.util.ArrayList**: lista de errores sintácticos y de símbolos.
- **java.util.Collections**: devolver las listas de la tabla de símbolos como vistas inmutables.

Análisis de resultados: objetivos alcanzados, objetivos no alcanzados, y razones por las cuales no se alcanzaron los objetivos (en caso de haberlos).

Objetivos alcanzados

- El lexer reconoce todos los tokens del lenguaje, incluyendo los símbolos raros que usa, y detecta errores léxicos sin detenerse: sigue escaneando y recopilándolos.
- La gramática quedó completa y sin ambigüedades, validada con la herramienta CUP.
- Cubre todo lo que pide el lenguaje: variables, arreglos, funciones con parámetros, condicionales, ciclos, operadores aritméticos, relacionales y lógicos. Lo probamos con un archivo grande que usa todo junto y pasó bien.

- Reporta los errores léxicos y sintácticos por separado, con la línea donde ocurrió cada uno, y dice correctamente si el archivo es válido o no.
- Si hay un error de sintaxis, el programa no se detiene de una vez: sigue revisando el resto del archivo y puede reportar varios errores en una sola corrida.
- La tabla de símbolos detecta nombres repetidos apenas se declaran, organizados por el ámbito donde están (global, función, bloque).

Objetivos no alcanzados

- Ninguno

Razones

- Ninguna

Justificación de toma de decisiones: por qué se generaron las producciones en expresiones y la precedencia.

Por qué la precedencia quedó así

Para que el exponente se resuelva antes que la suma y la resta, estas antes que la multiplicación y la división, y estas antes que el módulo, hicimos que cada nivel de precedencia fuera su propio no terminal, metidos uno dentro del otro, del más apretado al más suelto:

OPERANDO

→ `EXPR_POTENCIA`

→ `EXPR_SUMA_RESTA`

→ `EXPR_MULT_DIV`

→ `EXPR_ARITMETICA`

Cada nivel solo puede usar operandos del nivel de adentro, así que el parser está obligado a resolver primero lo que está más apretado antes de poder aplicar lo de afuera. Esto resulta más confiable que usar las declaraciones precedence de CUP, porque con los niveles anidados el conflicto directamente no existe. No es que CUP lo resuelva por su cuenta, es que la gramática ya no tiene ninguna ambigüedad que resolver.

Por qué no una sola regla tipo "expr OP expr" para todo

Se podría haber hecho una única producción genérica que agrupara todos los operadores juntos, pero eso es ambiguo por sí solo: no dice ni la precedencia ni cómo se agrupan los operadores, y todo quedaría a merced de que CUP lo adivine bien. Por eso cada operador quedó con su propia producción en su propio nivel. Y esto no se quedó solo en teoría: compilamos la gramática con CUP y dio 0 conflictos, ni un solo punto donde el parser tuviera que adivinar cómo agrupar algo.

Asociatividad

La suma, la resta, la multiplicación, la división y el módulo quedaron asociativos por la izquierda, porque la producción es recursiva hacia la izquierda, que es como normalmente se leen estas operaciones: $a \text{ menos } b \text{ menos } c$ se lee como $(a \text{ menos } b) \text{ menos } c$.

La potencia en cambio quedó asociativa por la derecha, que es como se acostumbra matemáticamente: $\text{dos elevado a tres elevado a dos}$ se interpreta como $\text{dos elevado a (tres elevado a dos)}$, no al revés.

Relacionales: por qué igual y diferente quedaron aparte

Al principio intentamos compartir un solo no terminal entre los seis operadores relacionales, pero eso generaba un conflicto real. Al cerrar el corchete, el parser no sabía si lo que acababa de leer iba para menor, menor o igual, mayor o mayor o igual, o si iba para igual y diferente, porque en ambos casos se llega al mismo punto con el mismo token de adelanto. El operador que realmente distingue el caso aparece después, cuando ya es tarde para decidir. La solución fue separarlos: igual y diferente quedaron con sus propias reglas explícitas, una para el caso numérico y dos más aparte para cuando se comparan booleanos o caracteres, sin pasar por ningún no terminal compartido. Así se quitó el conflicto sin perder la posibilidad de comparar booleanos y caracteres con estos dos operadores.

El paréntesis compartido entre aritmética y lógica

El mismo símbolo de paréntesis se usa tanto para agrupar una cuenta como para envolver un operando lógico, así que algo como (x) por sí solo resulta ambiguo. Con un solo token de adelanto el parser no puede saber si ese paréntesis es aritmético o lógico, porque los dos casos cierran igual y en los dos puede haber simplemente un identificador adentro. Por eso se creó un no terminal compartido, GRUPO_PARENTESIS, que usan tanto la agrupación aritmética como las expresiones lógicas. Cuál de los dos es en realidad, aritmético o lógico, se decide más adelante, no en esta etapa del análisis.

Bitácora

Registro de commits en el repositorio:

<https://github.com/MaikelFn/Proyecto-1-Compiladores>